

Mutation Testing Report

Subject: Java Project (math and io packages)

Tools Used: JUnit, PIT (PITest), Mockito

Overview

Mutation testing is used to evaluate the effectiveness of unit tests. It works by injecting small faults (mutations) into the code and verifying whether the test suite detects them. A mutation is "killed" if a test fails because of it, and "survived" if it does not.

Initial State

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
4	95% 55/58	77% 37/48	80% 37/46

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
io	1	88% 22/25	38% 5/13	45% 5/11
math	3	100% 33/33	91% 32/35	91% 32/35

Report generated by [PIT](#) 1.15.0

Enhanced functionality available at [arcmutate.com](#)

After taking some steps

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
4	100% 57/57	98% 45/46	98% 45/46

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
io	1	100% 24/24	91% 10/11	91% 10/11
math	3	100% 33/33	100% 35/35	100% 35/35

Report generated by [PIT](#) 1.15.0

Enhanced functionality available at arcmutate.com

Mutation Testing Summary

Package	Class	Total Mutations	Survived	Killed
math	ArithmeticOperations	13	0	13
math	ArrayOperations	5	0	5
math	MyMath	17	0	17
io	FileIO	11	1	10

Detailed Report

Pit Test Coverage Report

Package Summary

math

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
3	100% 33/33	100% 35/35	100% 35/35

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
ArithmeticOperations.java	100% 11/11	100% 13/13	100% 13/13
ArrayOperations.java	100% 7/7	100% 5/5	100% 5/5
MyMath.java	100% 15/15	100% 17/17	100% 17/17

Report generated by [PIT](#) 1.15.0

Mutations

24	1. negated conditional → KILLED
27	1. Replaced double division with multiplication → KILLED
	2. replaced double return with 0.0d for math/ArithmeticOperations::divide → KILLED
	1. negated conditional → KILLED
45	2. negated conditional → KILLED
	3. changed conditional boundary → KILLED
	4. changed conditional boundary → KILLED
47	1. negated conditional → KILLED
	1. Replaced integer division with multiplication → KILLED
49	2. negated conditional → KILLED
	3. changed conditional boundary → KILLED
50	1. Replaced integer multiplication with division → KILLED
	2. replaced int return with 0 for math/ArithmeticOperations::multiply → KILLED

Mutations

33	1. negated conditional → KILLED
	2. changed conditional boundary → KILLED
35	1. negated conditional → KILLED
40	1. replaced int return with 0 for math/ArrayOperations::lambda\$findPrimesInFile\$0 → KILLED
	2. replaced return value with null for math/ArrayOperations::findPrimesInFile → KILLED

Mutations

	1. negated conditional → KILLED
25	2. negated conditional → KILLED
	3. changed conditional boundary → KILLED
	4. changed conditional boundary → KILLED
30	1. changed conditional boundary → KILLED
	2. negated conditional → KILLED
31	1. Replaced integer multiplication with division → KILLED
35	1. replaced int return with 0 for math/MyMath::factorial → KILLED
48	1. changed conditional boundary → KILLED
	2. negated conditional → KILLED
51	1. changed conditional boundary → KILLED
	2. negated conditional → KILLED
	3. Replaced integer division with multiplication → KILLED
52	1. Replaced integer modulus with multiplication → KILLED
	2. negated conditional → KILLED
59	1. replaced boolean return with true for math/MyMath::isPrime → KILLED
	2. replaced boolean return with false for math/MyMath::isPrime → KILLED

math.ArithmeticOperations

Mutation: $x \leq \text{Integer.MAX_VALUE} / y \rightarrow x < \text{Integer.MAX_VALUE} / y$

Reason: Missing boundary case.

Fix:

```
@org.junit.Test
public void testMultiplyAtBoundary() {
    int x = Integer.MAX_VALUE / 2;
    int y = 2;
    assertEquals(x * y, new ArithmeticOperations().multiply(x, y));
}
```

Mutation: Integer overflow check

Fix:

```
@org.junit.Test(expected = IllegalArgumentException.class)
public void testMultiplyOverflowFromBoundaryCondition() {
    new ArithmeticOperations().multiply(Integer.MAX_VALUE / 2 + 1, 2);
}
```

math.MyMath

Mutation: Boundary check $n < 0 \parallel n > 12$

Fix:

```
@org.junit.Test
public void factorialOfZero() {
    assertEquals(1, new MyMath().factorial(0));
}

@org.junit.Test
public void factorialOfTwelve() {
    assertEquals(479001600, new MyMath().factorial(12));
}
```

Mutation: isPrime() loop condition change

Fix:

```
@org.junit.Test
public void isNotPrimeFour() {
    assertFalse(new MyMath().isPrime(4));
}
```

Pit Test Coverage Report

Package Summary

io

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	100% 24/24	91% 10/11	91% 10/11

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
FileIO.java	100% 24/24	91% 10/11	91% 10/11

Report generated by [PIT](#) 1.15.0

Mutations

40	1. removed call to java/io/PrintStream::println → KILLED
42	1. negated conditional → KILLED
43	1. removed call to java/io/PrintStream::println → KILLED
52	1. negated conditional → TIMED_OUT
58	1. removed call to java/io/PrintStream::println → KILLED
63	1. removed call to java/io/IOException::printStackTrace → SURVIVED
67	1. negated conditional → KILLED
68	1. removed call to java/io/PrintStream::println → KILLED
72	1. removed call to java/io/PrintStream::println → KILLED
73	1. replaced int return with 0 for io/FileIO::lambda\$readFile\$0 → KILLED
	2. replaced return value with null for io/FileIO::readFile → KILLED

Mutation Fixes:

```
@Test
public void testReadFileWithSingleLine() throws Exception {
    File tempFile = File.createTempFile("test", ".txt");
    tempFile.deleteOnExit();
    try (FileWriter writer = new FileWriter(tempFile)) {
        writer.write("42\n");
    }

    FileIO fileIO = new FileIO();
    int[] result = fileIO.readFile(tempFile.getAbsolutePath());
    assertEquals(new int[]{42}, result);
}
```

```
@Test
public void testReadFileLogsInputPath() throws Exception {
    File tempFile = File.createTempFile("logtest", ".txt");
    tempFile.deleteOnExit();
}
```

```

    try (FileWriter writer = new FileWriter(tempFile)) {
        writer.write("1\n");
    }

    FileIO fileIO = new FileIO();
    fileIO.readFile(tempFile.getAbsolutePath());

    assertTrue(systemOutRule.getLog().contains("==> readFile called with: " +
tempFile.getAbsolutePath()));
}

@Test
public void testOnlyInvalidLinesPrintsEmptyMessage() throws Exception {
    FileIO fileIO = new FileIO();
    URL resourceUrl = getClass().getClassLoader().getResource("only_invalid.txt");
    assertNotNull(resourceUrl);
    String path = Paths.get(resourceUrl.toURI()).toString();

    try {
        fileIO.readFile(path);
        fail("Expected IllegalArgumentException");
    } catch (IllegalArgumentException e) {
        assertEquals("Given file is empty", e.getMessage());
        assertTrue(systemOutRule.getLog().contains("==> File was empty"));
    }
}

```

In the test `testReadFileLogsInputPath`, the mutation on `if (!file.exists())` is killed because the test creates a real file, so the condition evaluates to false and the method continues normally. When PIT negates this condition, it incorrectly assumes the file doesn't exist and throws an exception, causing the test to fail and killing the mutation. The mutations that remove `System.out.println(...)` calls are also killed because the tests rely on console logs—specifically, `testReadFileLogsInputPath` and `testOnlyInvalidLinesPrintsEmptyMessage` check that certain messages are printed, such as the input path and "File was empty". If those prints are removed, the `systemOutRule` assertions fail. In the test `testOnlyInvalidLinesPrintsEmptyMessage`, the code throws an `IllegalArgumentException` when no valid numbers are parsed, which also leads to the killing of mutations related to the file being empty. On the other hand, the conditional negation mutation on line 52 times out, likely due to an infinite loop or incorrect handling of file lines. Finally, in `testReadFileWithSingleLine`, the returned array is asserted to contain the parsed number, so any mutation (like returning null or replacing values) will cause the test to fail and kill the mutation.

Conclusion

The mutation testing process successfully demonstrated the robustness and effectiveness of the unit tests written for the Java project's `math` and `io` packages. Out of a total of 46 mutations, 45 were killed, showing that the current test suite is highly effective at detecting incorrect or altered logic. Specifically, all mutations in `ArithmeticOperations`, `ArrayOperations`, and `MyMath` classes were killed, indicating comprehensive test coverage and strong assertion validation for mathematical operations and boundary conditions.

In the `FileIO` class, 10 out of 11 mutations were killed. The remaining survived mutation (`IOException.printStackTrace()` removal) points to a lack of coverage for real I/O error scenarios—such as simulating unreadable or corrupted files—which could be addressed in future tests, possibly using mocking. Tests like `testReadFileWithSingleLine`, `testReadFileLogsInputPath`, and `testOnlyInvalidLinesPrintsEmptyMessage` were instrumental in validating file reading logic, logging, and input validation, thereby eliminating most mutations in the `FileIO` class.

Overall, the mutation testing outcome confirms that the test suite is well-structured and catches a wide range of faults, with only minimal improvement needed for handling exceptional I/O cases. This reinforces the reliability of the tested codebase and highlights mutation testing as a powerful technique for evaluating test adequacy.